



2. Data Normalization Methods

Bringing Data to the Same Page by Scaling

Previous Section:

 [1. Data Preparation for Machine Learning](#)

Contents:

[1. Normalization - Putting your data into a specific scale](#)

[2. Min-Max Normalization](#)

[3. Z-Score Normalization \(Standardization\)](#)

[4. Normalization using Python](#)

[Summary](#)

[References](#)

Data is messy—let's make that clear first. And one of the biggest sources of that messiness is **inconsistency in scale**.

Imagine the following sequence:

[18.5, 13.4, 14.2, 15.3, 18.5, 19.1, 14.7, 15.2, 999.9, 22.4]

Let's say this represents BMI scores. At first glance, everything looks reasonable—most values fall within a similar range.

But then suddenly... there's 999.9.

That single value completely breaks the pattern. It doesn't belong to the same "world" as the rest of the data. And this creates problems.

- First, visualization becomes difficult. To include that one extreme value, your entire graph has to stretch up to nearly 1000—while most data sits quietly between 10 and 25.

- Second, statistical measures like the mean and even the median can become distorted, pulled toward that extreme point.

But underneath all of this, there's a deeper issue. The real problem is **the range of the data**.

- When values exist on completely different scales (commonly referred to as outliers), the dataset stops being consistent. Some numbers dominate, while others become almost invisible.

And that's exactly where normalization comes in.

1. Normalization - Putting your data into a specific scale

Normalization—at its core—is about **putting your data into a specific scale**.

In simple terms, normalization is any method used to **rescale data**. And when we say "scale," we mean transforming values so that they all fall within a certain range—an interval where everything is comparable.

This idea shows up everywhere: in data analysis, in visualization, and especially in machine learning—which is exactly what we care about in this course.

Now, let's make this concrete. Consider the following dataset:

Height (cm)	Weight (kg)	Resting Heart Rate (BPM)	Daily Calories (Est.)	Heart Disease
150	45	68	1393	0
155	50	81	1654	1
158	52	76	1617	1
160	54	72	1681	0
162	56	69	1816	0
165	58	82	1664	1
168	61	68	1779	0
170	63	80	1959	1
172	65	84	1978	1
175	68	72	2207	0

Height (cm)	Weight (kg)	Resting Heart Rate (BPM)	Daily Calories (Est.)	Heart Disease
178	70	72	2149	0
180	72	82	2162	1
182	74	65	2514	0
185	78	69	2473	1
188	80	64	2566	1

At first glance, everything looks fine.

But if you pay closer attention, each column lives in a **completely different range**:

- Height is around 150–190
- Weight is around 45–80
- Heart rate is around 60–85
- But calories? That jumps to 1000–2500+

Now here's the subtle problem. Even though logically these features might be equally important, **numerically they are not treated equally**.

- Models don't "understand meaning" the way humans do
- They see numbers—and numbers with larger magnitudes can dominate

So what happens?

- The model may **prioritize larger values over smaller ones**
- Features like calories can overshadow things like heart rate or weight
- In other words: **the bigger the number, the more influence it can have**

And that's not because it's more important—it's just because of its scale.

That imbalance is exactly what normalization is designed to fix.

So with that said, in this section, we're going to explore two of the most common normalization methods:

- **Min-Max Normalization**
- **Z-Score Normalization**

Let's start with the one that almost everyone encounters first.

2. Min-Max Normalization

Min-Max Normalization—also known as Min-Max Scaling—is one of the most intuitive and widely used scaling methods. In fact, when people first hear the word “*normalization*”, this is usually what they think of.

The idea is very simple:

- We take all the data
- And **rescale it into a fixed interval**, most commonly $[0, 1]$

Which means:

- The **smallest value** in the dataset becomes **0**
- The **largest value** becomes **1**
- And everything else is proportionally placed somewhere in between

So instead of having values like 13, 50, or 2000... everything is compressed neatly into a consistent range.

Mathematically, the transformation looks like this:

$$x_i' = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}}$$

Where:

- x_i is the original value
- $x_{\min}$ is the minimum value in the dataset
- $x_{\max}$ is the maximum value in the dataset
- x_i' is the normalized value

Now, let's go back to our earlier sequence. Technically, you *could* compute this by hand, but honestly—there's no real reason to.

So instead, let's use code. Here I'm using **pandas**, because honestly, I just love pandas.

```
import pandas as pd

sequence = [18.5, 13.4, 14.2, 15.3, 18.5, 19.1, 14.7, 15.2,
999.9, 22.4]
s = pd.Series(sequence, name='X')

# Convert to dataframe
df = s.to_frame()

# Apply Min-Max Scaling
df['X_scaled'] = [(df['X'].iloc[i] - df['X'].min()) / (df
['X'].max() - df['X'].min()) for i in range(len(df))]
print(df)
```

```
      X  X_scaled
0  18.5  0.005170
1  13.4  0.000000
2  14.2  0.000811
3  15.3  0.001926
4  18.5  0.005170
5  19.1  0.005778
6  14.7  0.001318
7  15.2  0.001825
8  999.9  1.000000
9   22.4  0.009123
```

Once we apply Min-Max Scaling, we get:

X	X_scaled
18.5	0.005170
13.4	0.000000
14.2	0.000811
15.3	0.001926
18.5	0.005170
19.1	0.005778
14.7	0.001318
15.2	0.001825
999.9	1.000000

X	X_scaled
22.4	0.009123

Now something very interesting happens here.

- The smallest value, 13.4, becomes exactly 0
- The largest value, 999.9, becomes exactly 1

Everything else is squeezed in between.

But look closely:

- Most of the “normal” values—those between 13 and 22—are now extremely close to 0
- While that single outlier, 999.9, completely dominates the scale

So even though we successfully normalized the data into the range $[0, 1]$, we didn’t actually solve the core problem.

We just **compressed the majority of the data into a tiny region near zero**.

Now, let’s visualize this. We’ll plot both the original values and the scaled values side by side, so you can clearly see how the distribution changes.

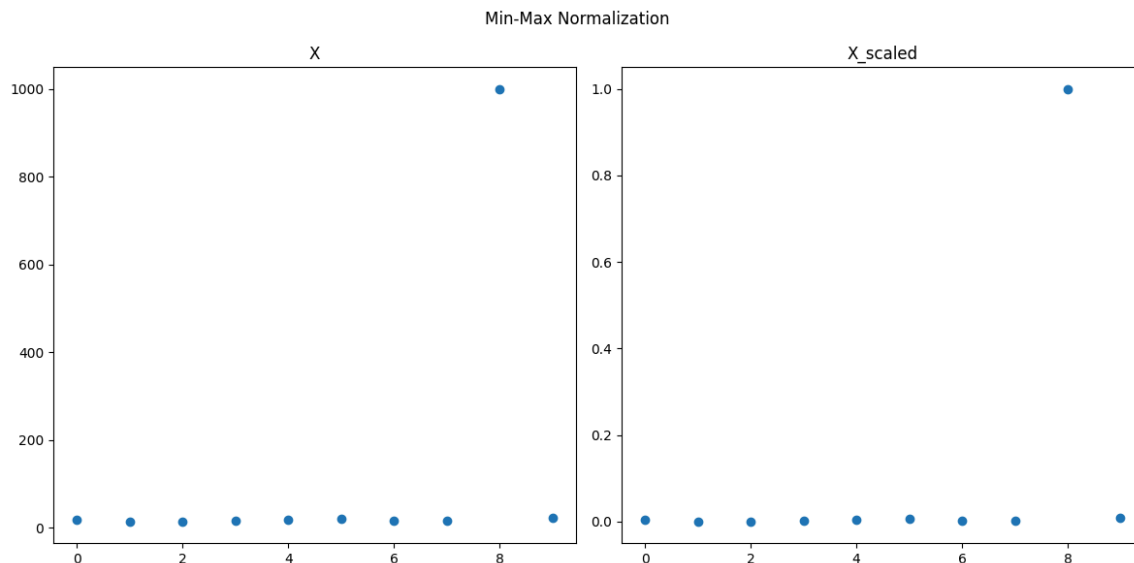
```
import matplotlib.pyplot as plt

# Plot a scatter plot for X and X_Scaled
# Draw two bar plots to compare the average GPA among all majors
fig, axs = plt.subplots(1, 2, figsize=(12, 6))
axs[0].scatter(df.index, df['X'], label='X')
axs[0].set_title('X')

axs[1].scatter(df.index, df['X_scaled'], label='X_scaled')
axs[1].set_title('X_scaled')

# Set super title
plt.suptitle('Min-Max Normalization')
```

```
plt.tight_layout()
plt.show()
```



And here's something important to notice:

- The **shape of the data does not change**
- The **relative positions between points stay exactly the same**

In other words, the two plots look **identical**. But there's one key difference:

- The **scale** is completely different
- The original data ranges from around **13** → **1000**
- The normalized data is compressed into **0** → **1**

And this reveals a very important idea: Normalization does **not** change the structure of the data. It only changes **how the data is represented numerically**.

Think of it like resizing an image: The picture stays the same. But the dimensions change

However... there's still a problem. That extreme value—999.9—is still dominating the scaling process. And as you saw, almost all other values are pushed very close to 0

So while Min-Max Normalization fixes the *range issue*, it is **very sensitive to outliers**.

And that leads us naturally to the next method, one that handles this situation much better: **Z-Score Normalization**.

3. Z-Score Normalization (Standardization)

Z-Score Normalization—also called **Standardization**—is another way to scale data. Or more precisely, to *standardize* it.

Now, technically, normalization and standardization fall under the same big idea: **rescaling data**. But the approach here is very different.

Instead of forcing values into a fixed range like $[0, 1]$, Z-Score Normalization transforms the data based on its **statistical properties**:

- The **mean (μ) becomes 0**
- The **standard deviation (σ) becomes 1**

So rather than asking: "Where does this value sit between the min and max?"

We ask: "**How far is this value from the average?**"

Mathematically, it looks like this:

$$z_i = \frac{x_i - \mu}{\sigma}$$

Where:

- x_i is the original value
- μ is the mean of the dataset
- σ is the standard deviation
- z_i is the standardized value

So what does this actually do?

- Values **above the mean** become positive
- Values **below the mean** become negative
- The farther away a value is from the mean, the larger its magnitude

This gives us something powerful:

- We're no longer just scaling data
- We're **measuring relative importance in terms of deviation**

And here's why many people prefer this method:

- It tends to **handle outliers better than Min-Max Scaling**
- It preserves the idea of **distance from the average**, which is very important in many models
- It feels more “statistical”—though that doesn’t automatically mean better

Now, let's go back to the exact same sequence. We'll apply this formula using code, visualize the result, and compare it directly with what we saw earlier.

Let's see how different the behavior really is.

```
import pandas as pd

sequence = [18.5, 13.4, 14.2, 15.3, 18.5, 19.1, 14.7, 15.2,
999.9, 22.4]
s = pd.Series(sequence, name='X')

# Convert to dataframe
df = s.to_frame()

# Apply Standardization
df['X_scaled'] = [(df['X'].iloc[i] - df['X'].mean()) / (df
['X'].std()) for i in range(len(df))]
print(df)
```

```
      X  X_scaled
0  18.5 -0.310783
1  13.4 -0.327187
2  14.2 -0.324614
3  15.3 -0.321076
4  18.5 -0.310783
5  19.1 -0.308853
6  14.7 -0.323006
7  15.2 -0.321397
8  999.9  2.845935
9  22.4 -0.298238
```

X	X_scaled
0	18.5

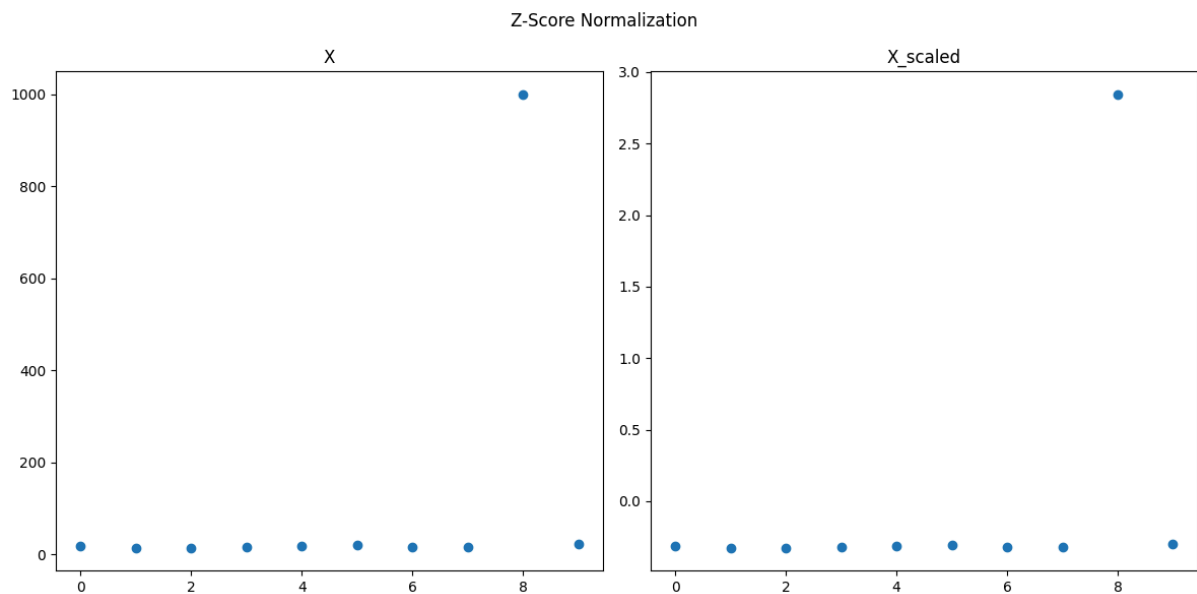
X	X_scaled
1	13.4
2	14.2
3	15.3
4	18.5
5	19.1
6	14.7
7	15.2
8	999.9
9	22.4

```
import matplotlib.pyplot as plt

# Plot a scatter plot for X and X_Scaled
# Draw two bar plots to compare the average GPA among all m
ajors
fig, axs = plt.subplots(1, 2, figsize=(12, 6))
axs[0].scatter(df.index, df['X'], label='X')
axs[0].set_title('X')

axs[1].scatter(df.index, df['X_scaled'], label='X_scaled')
axs[1].set_title('X_scaled')

# Set super title
plt.suptitle('Z-Score Normalization')
plt.tight_layout()
plt.show()
```



Something familiar happens again:

- The **shape of the data remains exactly the same**
- The **relative ordering between points does not change**

So visually, the pattern still looks identical. But here's where things become different. This time, the transformation is not based on **min and max**. It is based on the **mean and standard deviation**. Which means:

- The center of the data is now 0
- Most values are distributed around that center
- And distances are measured in terms of **how many standard deviations away** a point is

So instead of squeezing everything into $[0, 1]$, we now have a scale that looks more like:

- Negative values → below average
- Positive values → above average
- Larger magnitude → further from the mean

And here's the key observation:

- That extreme value (999.9) is still an outlier

- But it **doesn't compress the rest of the data into a tiny region** like Min-Max did

Instead:

- The normal values are still **spread out and distinguishable**
- The outlier simply appears as a **very large deviation**, rather than dominating the entire scale

So when you said: **"The difference here is the X-axis"**

That's exactly right.

- The **structure (shape)** of the data stays the same
- But the **interpretation of distance** completely changes

And that's the real takeaway:

- **Min-Max Scaling** focuses on *range*
- **Z-Score Normalization** focuses on *distribution*

Two different perspectives for solving the same fundamental problem: **making data comparable**.

4. Normalization using Python

Thankfully, the Python Scikit-Learn is now the most straightforward way to perform scaling. So that we do not have to remember the formulas every time (even though remembering those formulas is not a big deal).

```
import pandas as pd

# Sample dataset
data = {
    "Height (cm)": [150, 155, 158, 160, 162, 165, 168, 170,
172, 175, 178, 180, 182, 185, 188],
    "Weight (kg)": [45, 50, 52, 54, 56, 58, 61, 63, 65, 68,
70, 72, 74, 78, 80],
    "Resting Heart Rate (BPM)": [68, 81, 76, 72, 69, 82, 6
8, 80, 84, 72, 72, 82, 65, 69, 64],
    "Daily Calories (Est.)": [1393, 1654, 1617, 1681, 1816,
1664, 1779, 1959, 1978, 2207, 2149, 2162, 2514, 2473, 256
```

```
6],
    "Heart Disease": [0, 1, 1, 0, 0, 1, 0, 1, 1, 0, 0, 1,
0, 1, 1]
}
```

```
df = pd.DataFrame(data)
```

```
from sklearn.preprocessing import MinMaxScaler

# Scaling features using Min-Max
scaler1 = MinMaxScaler()
# Create a new dataframe
df_scaled1 = pd.DataFrame(scaler1.fit_transform(df.drop(columns=["Heart Disease"])),
                           columns=df.columns[:-1])
print(f"Min-Max Normalization:\n {df_scaled1}")
```

Min-Max Normalization:

Height (cm) Weight (kg) Resting Heart Rate (BPM) Daily Calories (Est.)

0	0.000000	0.000000	0.20	0.000000
1	0.131579	0.142857	0.85	0.222506
2	0.210526	0.200000	0.60	0.190963
3	0.263158	0.257143	0.40	0.245524
4	0.315789	0.314286	0.25	0.360614
5	0.394737	0.371429	0.90	0.231032
6	0.473684	0.457143	0.20	0.329071
7	0.526316	0.514286	0.80	0.482523
8	0.578947	0.571429	1.00	0.498721
9	0.657895	0.657143	0.40	0.693947
10	0.736842	0.714286	0.40	0.644501
11	0.789474	0.771429	0.90	0.655584
12	0.842105	0.828571	0.05	0.955669
13	0.921053	0.942857	0.25	0.920716
14	1.000000	1.000000	0.00	1.000000

```

from sklearn.preprocessing import StandardScaler

scaler2 = StandardScaler()
df_scaled2 = pd.DataFrame(scaler2.fit_transform(df.drop(columns=["Heart Disease"])),
                           columns=df.columns[:-1])

print(f"Z-Score Normalization:\n {df_scaled2}")

```

Z-Score Normalization:

Height (cm) Weight (kg) Resting Heart Rate (BPM) Daily Calories (Est.)

0	-1.789992	-1.767659	-0.864373	-1.666554
1	-1.339490	-1.278454	1.142207	-0.918068
2	-1.069190	-1.082772	0.370446	-1.024175
3	-0.888989	-0.887091	-0.246964	-0.840638
4	-0.708789	-0.691409	-0.710021	-0.453489
5	-0.438488	-0.495727	1.296560	-0.889390
6	-0.168187	-0.202204	-0.864373	-0.559597
7	0.012013	-0.006523	0.987855	-0.043399
8	0.192214	0.189159	1.605265	0.011089
9	0.462515	0.482682	-0.246964	0.667807
10	0.732815	0.678363	-0.246964	0.501477
11	0.913016	0.874045	1.296560	0.538758
12	1.093216	1.069727	-1.327430	1.548211
13	1.363517	1.461091	-0.710021	1.430633
14	1.633818	1.656772	-1.481783	1.697335

Summary

Before I summarize, let's compare Min-Max Normalization and Z-Score Normalization side-by-side again.

- **Similarities:** At their core, both methods aim to do the same thing:
 - They **scale the data** to make features comparable
 - They **preserve the shape** of the data → plots look identical
 - They are commonly used in **distance-based models** like: KNN, Linear Regression, and K-Means

And here is a very important rule—one that beginners often miss:



You should **only apply normalization to numerical, non-target features**

Which means:

- **Numerical** → because scaling only makes sense for numbers
- **Non-target** → because the target variable is what you are trying to predict, not something you should distort

Another important note is that normalization must be applied consistently to both the training and testing data.

- If you normalize a feature during training, you must apply the exact same transformation when working with testing data or making predictions.
- This means:
 - Using the same method and the same parameters such as: T
 - The min and max values for Min-Max Scaling, or;
 - The mean and standard deviation for Z-Score.
 - Otherwise, the model will see data in a different scale than what it was trained on, which can lead to incorrect or unreliable predictions.

In short: They don't change the data's structure—only its **numerical representation**.

- **Differences**

Aspect	Min-Max Normalization	Z-Score Normalization
Scaling Idea	Based on range (min → max)	Based on distribution (mean & std)
Output Range	Fixed: $[0, 1]$	Unbounded (can be negative or positive)
Sensitivity to Outliers	Very sensitive	More robust
Best For	Data without extreme outliers	Data with normal-like distribution
Interpretation	Relative position in range	Distance from the mean

Aspect	Min-Max Normalization	Z-Score Normalization
Plot Difference	Same shape, different scale	Same shape, different X-axis

Other Normalization Methods: Of course, these are not the only ones.

There are other techniques like:

- Log Transform
- Robust Scaling
- Decimal Scaling

Each comes with its own assumptions and use cases.

This section ends somewhat abruptly—but very important: *Normalization is the process of **bringing data into a comparable scale**. It prevents features with large values from **dominating the model**. It ensures that models focus on **patterns**, not raw magnitude*

And perhaps most importantly: **Normalization does not change what the data is**. It only changes **how the data is seen by the model**.

And in Machine Learning, how the model sees the data is everything.

References

<https://www.geeksforgeeks.org/machine-learning/what-is-data-normalization/>

<https://developers.google.com/machine-learning/crash-course/numerical-data/normalization>

<https://www.datacamp.com/tutorial/normalization-in-machine-learning>

<https://www.codecademy.com/article/min-max-zscore-normalization>

Next Section:

 3. One-Hot Encoding